SOFTWARE REQUIREMENTS SPECIFICATION

for

PROJECT TEAM FORMATION PORTAL

Version 1.0

Prepared by : Shreyansh (1024240046)
Akshin (1024240036)
Chandan (1024240147)
Ayush (1024240062)

Submitted to : Dr Anushka
Lab Instructor

October 13, 2026

Contents

1 Introduction **3**
- 1.1 Purpose . . . 3
- 1.2 Intended Audience and Reading Suggestions . . . 3
- 1.3 Project Scope . . . 3

2 Overall Description **5**
- 2.1 Product Perspective . . . 5
- 2.2 User Classes and Characteristics . . . 5
- 2.3 Product Functions . . . 5
- 2.4 Operating Environment . . . 6
- 2.5 Design . . . 6

3 System Features **8**
- 3.1 Description and Priority . . . 8
- 3.2 Functional Requirements . . . 8

4 Other Nonfunctional Requirements **9**
- 4.1 Performance Requirements . . . 9
- 4.2 Security Requirements . . . 9
- 4.3 Software Quality Attributes . . . 9

5 Other Requirements **10**

1 Introduction

1.1 Purpose

The Project Team Formation Portal is a web application that helps students find teammates for academic projects and hackathons. It allows students to create a profile, show their skills, view project posts, and send requests to join teams. It also helps teachers publish opportunities and review student submissions.

1.2 Intended Audience and Reading Suggestions

This SRS is intended for the project team, faculty members, students, testers, and future maintainers. It explains the users and system context. It describes the main features. It lists quality and security requirements.

1.3 Project Scope

The portal provides one place for posting projects, discovering suitable team members, forming teams, and sharing basic project updates. Students can browse posts by type or required skill. Teachers can create and manage posts for projects or hackathons.

Figure 1.1 should show the flow from a teacher creating a post, to students applying, forming a team, and publishing the project update.

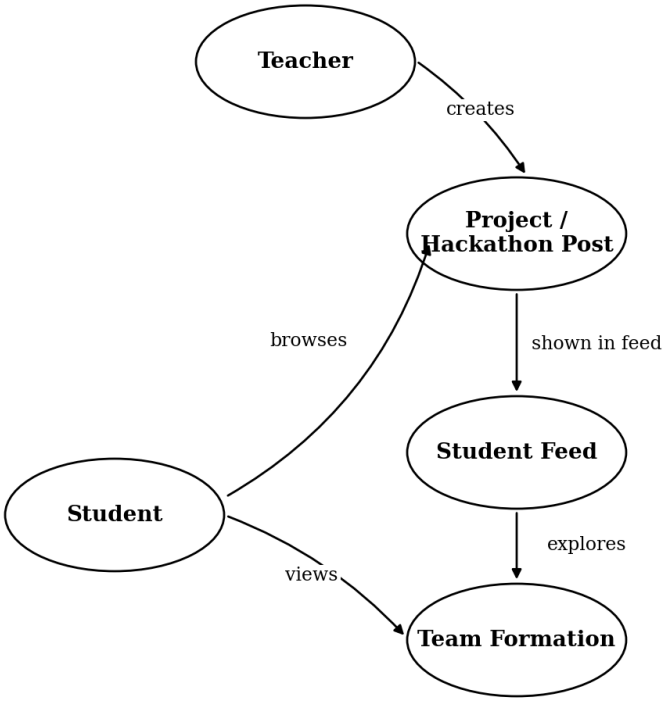


Figure 1.1: Overall workflow of the Project Team Formation Portal

2 Overall Description

2.1 Product Perspective

The portal is a standalone web application with a frontend, backend API and database. It replaces informal team formation through messages and notices with a searchable system. The frontend communicates with the backend through secure web requests, while the database stores user, project, team, and post data.

2.2 User Classes and Characteristics

The system has the following user classes:

- **Students:** create profiles, explore posts, apply to teams, and share projects.
- **Teachers:** create project or hackathon posts, view student submissions, and manage their posts.
- **Administrator:** maintains users, posts, and reported content.

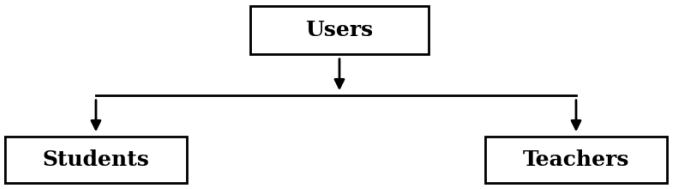


Figure 2.1: User roles and their main permissions

2.3 Product Functions

The main functions are registration and login, profile management, project-post creation, project browsing, skill-based search, join requests, team management, and a student feed. A student profile includes name, email, skills, interests, bio, and links. A project post includes title, type, description, required skills, owner, team size, and creation date.

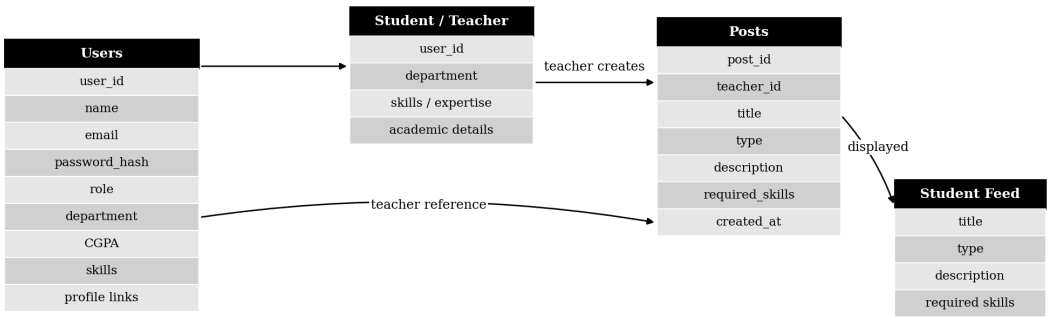


Figure 2.2: Data flow diagram of the portal

2.4 Operating Environment

The system will run in modern web browsers on Windows, Linux, macOS, Android, and iOS. The server will run on a Node.js-supported environment with MongoDB. Internet access is required to use the portal.

2.5 Design

The interface contains separate dashboards for students and teachers. The student dashboard presents profiles, posts, applications, and a feed. The teacher dashboard presents a form for creating posts and a list of posts created by that teacher.

Student activities include registering, adding skills, viewing posts, sending a join request, accepting team members, and uploading a project update.

Teachers create posts, edit or delete their own posts, and review submissions. Administrators may remove inappropriate content and manage user roles.

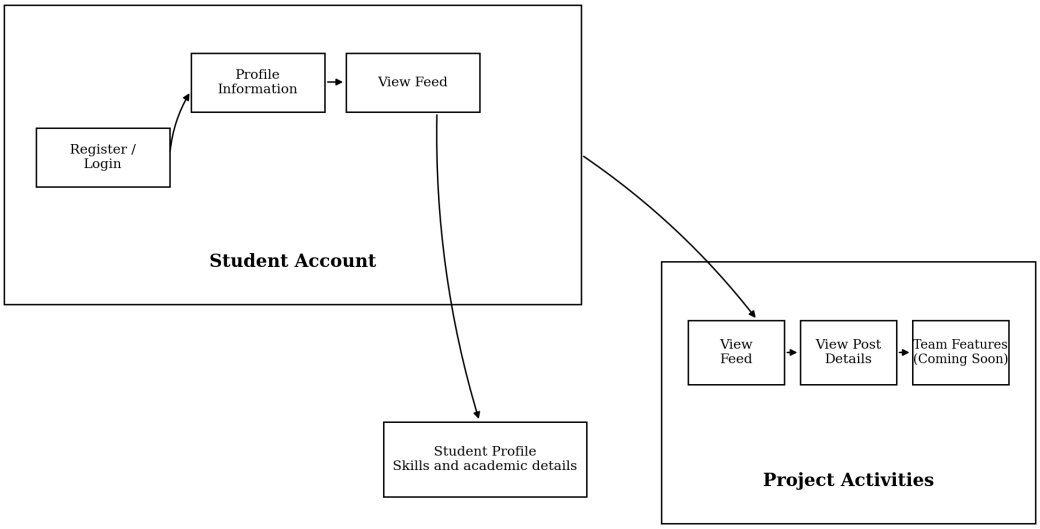


Figure 2.3: Student activities

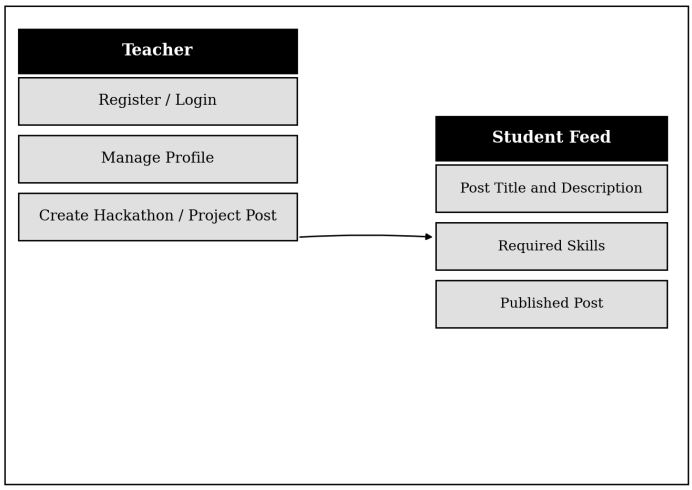


Figure 2.4: Teacher activities

3 System Features

3.1 Description and Priority

The features are listed in priority order:

1. **Authentication and roles:** users register, log in, and access functions according to their role.
2. **Student profiles:** students add skills, interests, and a bio.
3. **Project posts:** teachers create project and hackathon posts.
4. **Team formation:** students request to join and owners manage team membership.
5. **Feed and updates:** approved users create and view project updates.

3.2 Functional Requirements

1. The system shall allow students and teachers to register and log in with a valid email and password.
2. The system shall allow an authenticated user to view and update the profile fields permitted for that user.
3. The system shall allow a teacher to create, edit, and delete only the posts created by that teacher.
4. The system shall display project posts to students and allow filtering by post type and required skill.
5. The system shall allow a student to send a join request to a project team and allow the team owner to accept or reject it.
6. The system shall prevent a student from joining the same team twice.
7. The system shall allow authorized users to create project updates for the student feed.
8. The system shall show clear success and error messages for user actions.

The planned technology stack is React and JavaScript for the frontend, Node.js and Express for the backend, and MongoDB for the database.

4 Other Nonfunctional Requirements

4.1 Performance Requirements

For normal college use, common pages should load within three seconds on a stable internet connection. Searching and filtering posts should return results quickly for the expected number of students and posts.

4.2 Security Requirements

Passwords shall be stored in hashed form. Protected pages shall require login. Role checks shall prevent students from using teacher functions and prevent users from editing another user's protected data. Inputs shall be validated on the server before they are stored.

4.3 Software Quality Attributes

The portal should be easy to learn, responsive on common screen sizes, and maintainable through separate frontend and backend modules. Error messages should be understandable. The system should be tested for registration, login, post creation, filtering, and team-request workflows.

5 Other Requirements

The portal requires regular backup of the database and maintenance of deployed software. Future versions may add notifications, chat, recommendation of teammates, and faculty approval of completed teams.